pip install torch, torchvision

Ch 10.2: Multi-Layer Neural Nets

Lecture 29 - CMSE 381

Prof. Elizabeth Munch

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Wed, Nov 30, 2022

Announcements

Last time:

- Single Layer Neural Nets

This lecture:

- Multi-layer Neural Nets
- Application to MNIST

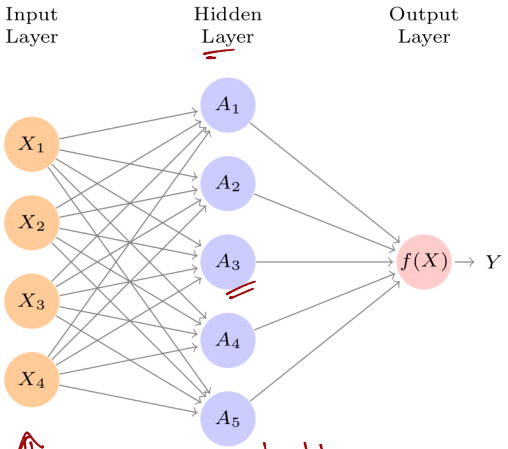
Announcements:

- HW #10

Section 1

Neural Nets

Feed Forward Neural Network: The cartoon



hidden unit

activation function

$$A_k = h_k(X) = g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j),$$

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

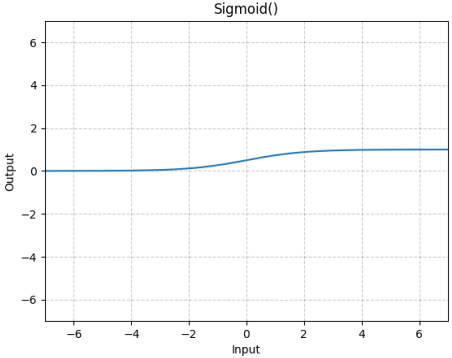
Architecture: all choices made before training

Choices for activation function

Sigmoid:

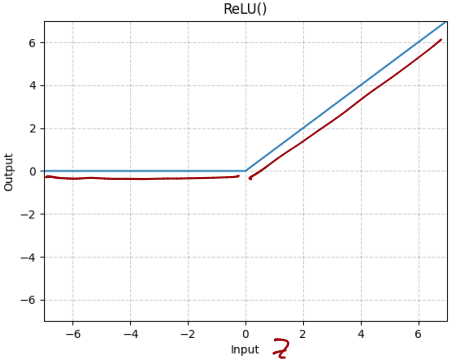
$$g(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

values in [0,1]

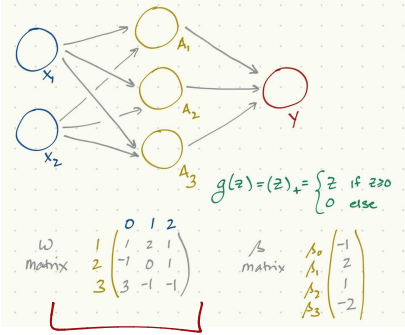


ReLU: Rectified linear unit

$$g(z) = (z)_+ = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{else.} \end{cases}$$



Matrix version



$$A_k = h_k(X) = g(w_{k0} + \textstyle\sum_{j=1}^{p} w_{kj} X_j),$$

$$A = g(\mathbf{W} \cdot \mathbf{X}) \qquad \mathbf{X}^T = (1\ X_1\ X_2\ \cdots\ X_p)$$

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

$$Y = \beta \cdot \mathbf{A} \qquad \mathbf{A}^T = (1\ A_1\ A_2\ \cdots\ A_K)$$

Vector $\{\beta_0, \beta_1, \ldots, \beta_K\}$

Training the model

Choose parameters by minimizing RSS, $\sum_{i=1}^{n}(y_i - f(x_i))^2$ (or other loss function)

Chosen in advance:

# of layers

# of hidden units

activation function

Tuned by the model:

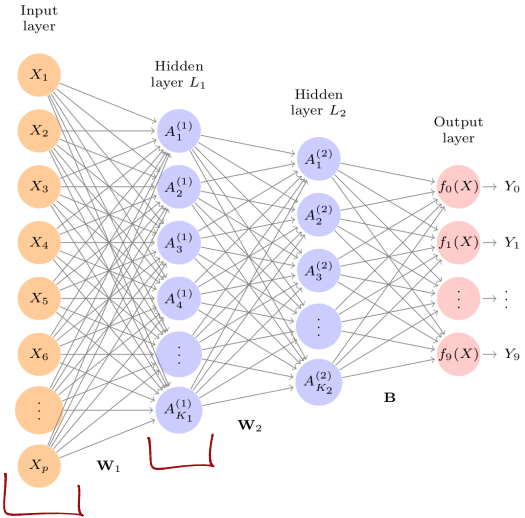
W

β

Section 2

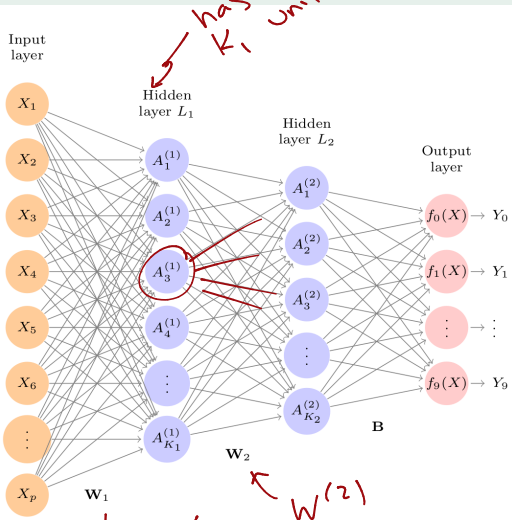
Multilayer Neural Networks

Multiple layers



- each new layer is linear combo of previous layer w/ activation function
- can have diff # of units in each layer

Hidden layers



has K_1 units

$W^{(1)}$

$W^{(2)}$

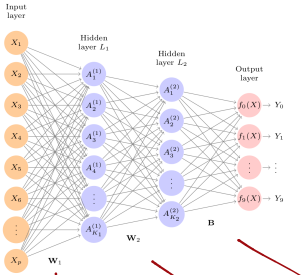
entry in layer 1

$$A_k^{(1)} = h_k^{(1)}(X) = g(w_{k0}^{(1)} + \sum_{j=1}^{p} w_{kj}^{(1)} X_j)$$

$$A_\ell^{(2)} = h_\ell^{(2)}(X) = g(w_{\ell 0}^{(2)} + \sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)})$$

entry for layer 2

More on that architecture

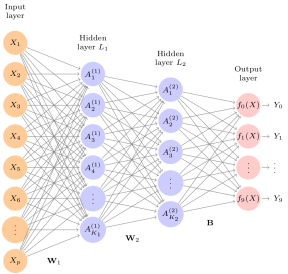


$$
\begin{aligned}
A_k^{(1)} &= h_k^{(1)}(X) \\
&= g(w_{k0}^{(1)} + \textstyle\sum_{j=1}^{p} w_{kj}^{(1)} X_j) \\
A_\ell^{(2)} &= h_\ell^{(2)}(X) \\
&= g(w_{\ell 0}^{(2)} + \textstyle\sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)})
\end{aligned}
$$

Have a lot of weights to be trained

matrix $p \times K_1$ $K_1 \times K_2$ $K_2 \times 10$

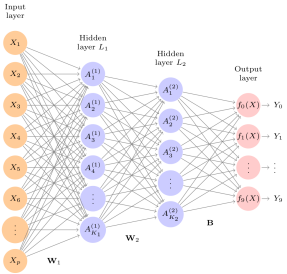
Matrix version: First layer



$$
\begin{aligned}
A_k^{(1)} &= h_k^{(1)}(X) \\
&= g(w_{k0}^{(1)} + \textstyle\sum_{j=1}^{p} w_{kj}^{(1)} X_j)
\end{aligned}
$$

$$A^{(1)} = g(\mathbf{W}^{(1)} \cdot \mathbf{X}) \qquad \mathbf{X}^T = (1\ X_1\ X_2\ \cdots\ X_p)$$

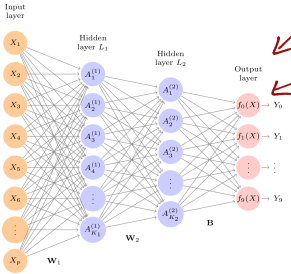
Matrix version: Second layer



$$
\begin{aligned}
A_\ell^{(2)} &= h_\ell^{(2)}(X) \\
&= g(w_{\ell 0}^{(2)} + \textstyle\sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)})
\end{aligned}
$$

$$
A^{(2)} = g(\mathbf{W}^{(2)} \cdot \mathbf{A}) \qquad (\mathbf{A}^{(1)})^T = (1 \; A_1^{(1)} \; A_2^{(1)} \; \cdots \; A_{K_1}^{(1)})
$$

Matrix version: Last layer, first step



for classification

one per label to predict

$$\underbrace{Z_m = \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} h_\ell^{(2)}(X)}$$

$$= \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} A_\ell^{(2)},$$

$$\mathbf{Z} = \beta \cdot \mathbf{A}$$

β is $\underbrace{M \times (K_2+1)}$ matrix $\qquad (\mathbf{A}^{(2)})^T = (1\ A_1^{(2)}\ A_2^{(2)}\ \cdots\ A_{K_2}^{(2)})$

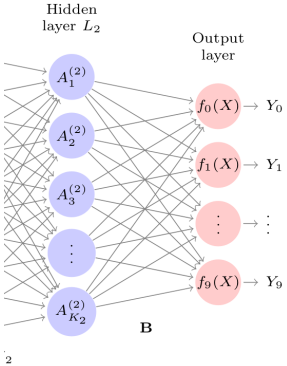
Have numbers as output

but I want probability of each label

The last column for classification: Softmax

Want: · each $f_m(X)$ to be in $[0,1]$

· Sum $\sum_{m=1}^{M} f_m(X) = 1$



the one I'm interested in now

$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{\ell=0}^{\cancel{9}M} e^{Z_\ell}},$$

Sum of all label entries

An example

dog cat clown pig sparrow

$$Z = \begin{pmatrix} z_1 & z_2 & z_3 & z_4 & z_5 \\ 1 & 3 & -1 & 2 & 5 \end{pmatrix}$$

$$f_2(x) = \frac{e^3}{e^1 + e^3 + e^{-1} + e^2 + e^5} = 0.1122$$

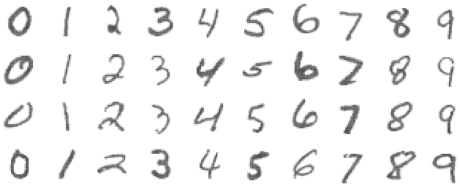
"cat"

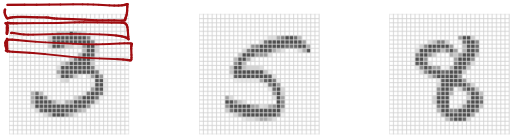
sparrow

Output vector $(0.015, 0.112, 0.002, 0.041, 0.829)$

What label to predict: the highest!

MNIST





28x28 pixel images

pixels are greyscale [0, 255]

Goal: predict the label
= the digit 0, 1, ..., 9

Convert the data to vector
flatten!

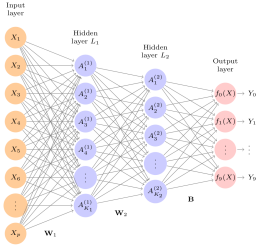
p = 784 vector

Coding

remote.egr.msu.edu

TL;DR

Feed Forward Neural Net



$$A_k = h_k(X) = g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j),$$

- Combines input data using learned weights
- Linear combo of those to get output
- Sometimes softmax to get probability of classification

Next time

20	F	Nov 4	Polynomial & Step Functions.	7.1,7.2	
21	M	Nov 7	Step Functions	7.2	
22	W	Nov 9	Basis functions, Regression Splines	7.3,7.4	
23	F	Nov 11	Decision Trees	8.1	HW #7 Due
24	M	Nov 14	Random Forests	8.2.1, 8.2.2	
25	W	Nov 16	Maximal Margin Classifier	9.1	
26	F	Nov 18	SVC	9.2	HW #8 Due
27	M	Nov 21	SVM	9.3, 9.4, 9.5	
28	W	Nov 23	Extended virtual office hours		
	F	Nov 25	No class - Thanksgiving		
29	M	Nov 28	Single layer NN	10.1	HW #9 Due
30	W	Nov 30	Multi Layer NN	10.2	
31	F	Dec 2	CNN	10.3	
32	M	Dec 5	Unsupervised Learning & Clustering	12.1, 12.4	HW #10 Due
	W	Dec 7	Review		
	F	Dec 9	Midterm #3	Bring your cheat sheet and a non-internet-connected calculator	